DSDB FPGA Implementation Guide

Instructor: Shahriar Shahabuddin
TAs: Hridi Prova Debnath/Md Salman Sakib/KM Faisal

This document will explain how to start using Xilinx Vivado to get Lab 2 working on the National Instrument (NI) Digital System Development Board's (DSDB's) FPGA [1]. The steps listed here will also work for future labs with some small caveats related to connection of devices on the DSDB in which implementation on the FPGA will also be required.

1. DSDB FPGA : Zynq XC7Z020-1CLG484C

The DSDB has a plethora of devices on it including the main chip we will use this semester, the AMD (formerly Xilinx) Zynq FPGA. The AMD Zynq XC7Z020-1CLG484C is extremely powerful sporting 650 MHz dual-core Cortex-A9 processors, DDR3 memory controller with 8 DMA channels, high-bandwidth peripheral controllers: 1G Ethernet, SDIO, and many others. More information on the DSDB can be found here: `https://reference.digilentinc.com/dsdb/rm` and on Canvas.

All Zynq devices have the same basic architecture, and all of them contain, as the basis of the processing system, a dual-core ARM Cortex-A9 processor. This is a 'hard' processor (PS) and it exists as a dedicated and optimized silicon element on the device. The second principal part of the Zynq architecture is the programmable logic (PL). This is based on the Artix®-7 and Kintex®-7 FPGA fabric. The PL is predominantly composed of general purpose FPGA logic fabric, which is composed of slices and Configurable Logic Blocks (CLBs), as well as Input/Output Blocks (IOBs) for interfacing (i.e., these are all AMD-specific terms). As explained in class, the CLBs are small, regular groupings of logic elements that are laid out in a two-dimensional array on the PL, and connected to other similar resources via programmable interconnects. Each CLB is positioned next to a switch matrix and contains two logic slices. Although today's FPGAs contain more devices other than CLBS (e.g., Block RAM), we will primarily use the CLBs for our designs. These collection of devices on the FPGAs are typically called System on Chip or SoC devices.

2. FPGA Implementation

To begin working with the Zynq platform, you will need to use the Vivado Electronic Design Automation (EDA) toolset from AMD. Before attempting implementation, it is absolutely essential that your design functions correctly in Questa/ModelSim simulation. Proceeding without a fully validated simulation greatly increases the likelihood of failure.

Do not move forward with the implementation steps until your Questa/ModelSim simulations are fully passing to the best of your knowledge. Additionally, a strong grasp of verification using a SystemVerilog testbench is critical for ensuring a functional FPGA implementation. Debugging fundamental design flaws in Vivado and on the FPGA is significantly more challenging compared to detecting and resolving them within Questa/ModelSim simulations.

Vivado is a powerful EDA tool available in the ENDV 350 laboratories. While it can be installed on personal machines, doing so may not be necessary unless you wish to explore its capabilities further.

2.1 Vivado System Design

Modern system design emphasizes design reuse and rapid development, as reducing time to market is crucial in many applications. Tools that accelerate the development process—without compromising verification integrity or design quality—offer clear advantages.

The Vivado design flow is built around these principles, leveraging pre-verified Intellectual Property (IP) blocks for efficient system integration. Unlike traditional design methodologies that require the building of systems from the ground up, Vivado prioritizes IP reuse from its built-in libraries, third-party vendors, and user-defined components. This approach shifts the primary focus to system integration rather than low-level hardware design, streamlining development without sacrificing flexibility.

That said, custom logic can still be integrated into the design, which is precisely what we will be doing with our Zynq FPGA. However, we will also leverage Vivado's IP catalog to ensure correct functionality and seamless integration with the DSDB (Digital System Development Board). This hybrid approach allows both efficiency and customization, ensuring a robust and functional design.

3. Setting Up the Vivado Project for the First Time

There is a `lab2` project that contains a pre-made Vivado project in the canvas.

1. Download the Lab2.zip from canvas and your Vivado project should be inside the `lab2/lab2` Vivado folder. Place the `setup.tcl` file from the `scripts` subdirectory of your lab2 repository on the desktop. Place the Lab 2 project folder wherever is convenient for you.

2. Place your `riscv_single.sv` file on the desktop that you wish to implement on the FPGA. Make sure you remove the `top` module from the `risc_single.sv` file that you want to implement before you go to the next section. This is done because we are substituting a version for the implementation that properly interfaces to the DDR3 memory controller.

3. You will need to modify your logic in your microarchitecture to output a `MemStrobe` whenever you are performing a Load or Store operation (e.g., `lw` or `sw` or `lb`) and stall the next instruction during this strobe. This can easily be done by asserting a signal in the `maindec` module of the `risc_single.sv` file. This can be seen in Figure 1. To stall the next instruction properly, the best option is to enable

```
assign {RegWrite, ImmSrc, ALUSrc, MemWrite,
        ResultSrc, Branch, ALUOp, Jump, MemStrobe} = controls;

always_comb
  case(op)
    // RegWrite_ImmSrc_ALUSrc_MemWrite_ResultSrc_Branch_ALUOp_Jump_MemStrobe
    7'b0000011: controls = 12'b1_00_1_0_01_0_00_0_1; // lw
    7'b0100011: controls = 12'b0_01_1_1_00_0_00_0_1; // sw
    7'b0110011: controls = 12'b1_xx_0_0_00_0_10_0_0; // R-type
    7'b1100011: controls = 12'b0_10_0_0_00_1_01_0_0; // beq
    7'b0010011: controls = 12'b1_00_1_0_00_0_10_0_0; // I-type ALU
    7'b1101111: controls = 12'b1_11_0_0_10_0_00_1_0; // jal
    default: controls = 12'bx_xx_x_x_xx_x_xx_x_x; // ???
  endcase // case (op)
```

Figure 1: Strobe Signal for MemStrobe

   the PC register. This can be done via the Enable signal in the register. An example of this is shown here:

```
flopenr #(32) pcreg (clk, reset, PCReady, PCNext, PC);
```

4. Open the Vivado project by first opening Vivado and then clicking Open Project under Quick Start (i.e., Figure 2). The Vivado project should be available in your directory. Navigate to the Lab 2 project folder you extracted from you Canvas zip file and select the Lab 2 project file inside that folder. You may need to click **yes** to allow the project to be updated your version of Vivado too. AMD's Vivado utilizes a lot of memory space, so it may take some time to open.

5. Before you implement your design, remove the `memfile.dat` and replace it with one of your programs (e.g., `test_hw.memfile`). To make things easier, copy your new memfile to `memfile.dat` and add it to your design via the "+" key under Sources.

   Just like a terminal, the Tcl Console runs inside a specific directory, but you can change that directory using the `cd` command. First, use the command `pwd` to Print your Working Directory (where you currently are). Using that information, navigate to your desktop by typing `cd ..` to ascend a folder level and `cd <name of folder>` to descend into that folder. Your goal here is to navigate to the desktop where we put the `setup.tcl` and `riscv_single.sv` files.

   - Beginning running the project toolchain which will start with Block Generation in which each top

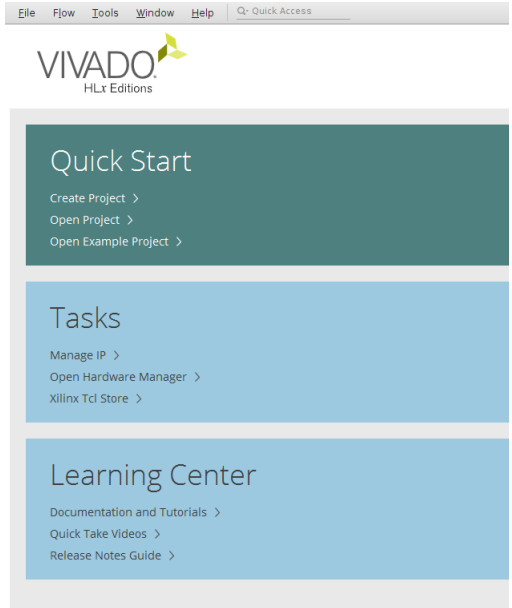

Figure 2: Quick Start and Open Project

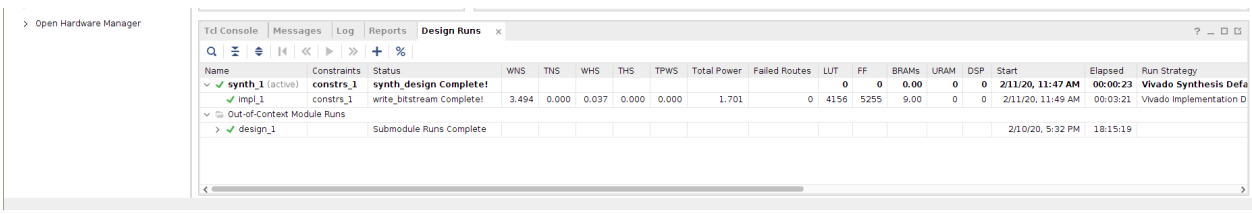

Figure 3: Bottom Panel with Tcl Console, Messages, etc.

level module in the project will be synthesized to check for usability. Then, Synthesis will convert the entire design and all surrounding components into a netlist that represents real hardware. Following that, Implementation will convert the hardware netlist into a design the FPGA can understand. Finally, Generate Bitstream will create a file that can be uploaded to the FPGA and allow the FPGA to start using your design.

Again, this is why you do not want to start using the FPGA until your simulation works - errors here take longer to find and are often more complicated to solve than if you had found them in your simulations in Questa/ModelSim. These steps might take some time to run (approximately 5 minutes) - you can view their progress in the Design Runs tab or looking at the upper-right hand corner of your Vivado screen.

If you run into errors or issues here, go to the Messages tab in where messages, warnings, and errors are stored and see what happened, or delve even deeper in the Log tab where the output from the toolchain is directly stored.

If you need to make modifications to your files, you will need to do so in Vivado now because **the script will copy the file on your desktop into the project directory**. Modifications made to the file on your desktop will not be reflected in the project. Go to Open Block Design on the left panel and then select the Sources tab in the upper left window (i.e., Figure 4). You'll have to do a bit of digging through the source tree to get to your RISC-V core because this project has many components in the background to make things run, expand the design_1_wrapper, design_1, top_0, design_1_top_0_0, and finally inst : top to reveal your RISC-V core called `riscvsingle:riscvsingle`. Double-click that entry to open the file. Make your changes and save the file.

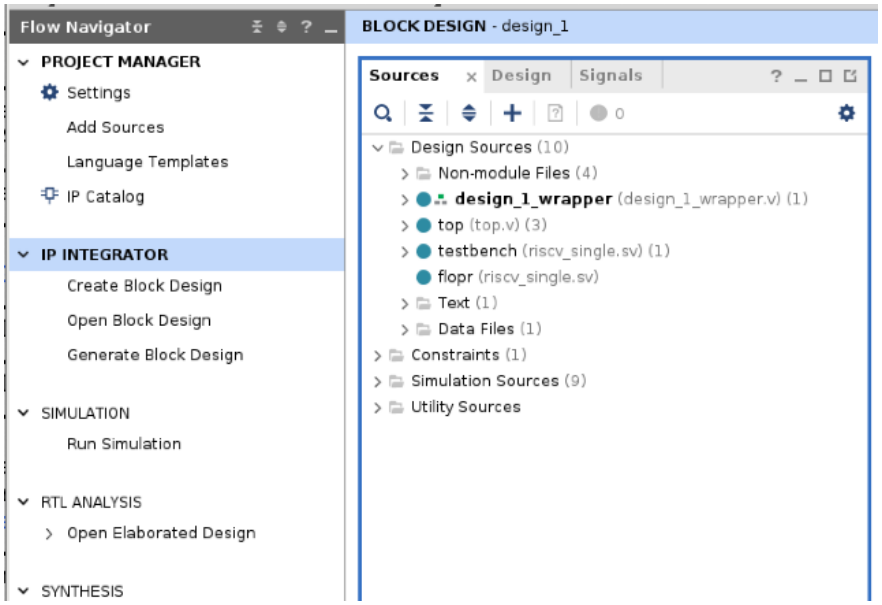

Figure 4: Open Block Design and Sources Window

Anytime you make a change to the file and save it, a message will appear along the top of the screen that the Module References are Out-of-Date (i.e., Figure 5). Like what the script was doing, you've changed the module, but the other modules that connect to the RISC-V core haven't had a chance to refresh themselves and update their references. Clicking Refresh Changed Modules on that message will update the project index and refresh all references. You can then run the toolchain just like the script did by clicking Generate Bitstream on the bottom-left panel. A window will pop up letting you know that Synthesis is out-of-date and that will need to run first - answer Yes so all required steps will run first. Then, the Launch configuration window will open where you can set details about how the toolchain will run. Leave these at the defaults and click OK. Go to the Design Runs tab in Figure 3 to see each step's progress while waiting for them to complete (approximately 5 minutes).

6. Once Bitstream Generation is complete, a window will pop up asking what you would like to do next (i.e., Figure 6). Select Open Hardware Manager and click OK. If you accidentally close this window or select something else, you can easily get to the hardware manager using the Open Hardware Manager button below Generate Bitstream.

7. If you have not already done so, now is the time to connect your DSDB to the computer and power it on, as the next step will involve opening a connection to it in Vivado so you can upload your design to the FPGA.

   Remove the prototyping board from the Elvis III at your desk by pulling the board horizontally towards you. Connect the DSDB by lining up the gold fingers along the top of the board with the socket on the top of the Elvis III. Make sure the board is laying flat before pushing the board into the socket - the holes in the PCB along the bottom should slide into the plastic notches along the bottom of the Elvis III. Connect the DSDB to the computer at your station using the microUSB to USB cable included in it's box.

   Yes, even though the DSDB is connected to the Elvis III, which is itself connected to the computer already, you still need to connect the DSDB to the computer with the USB cable in order to interact with the FPGA through Vivado.

   The DSDB will be powered by the connector along the top of the board through the Elvis III - you do not need to power it with another power source. Do <u>not</u> connect a power supply to the barrel connector on the top right of the DSDB.

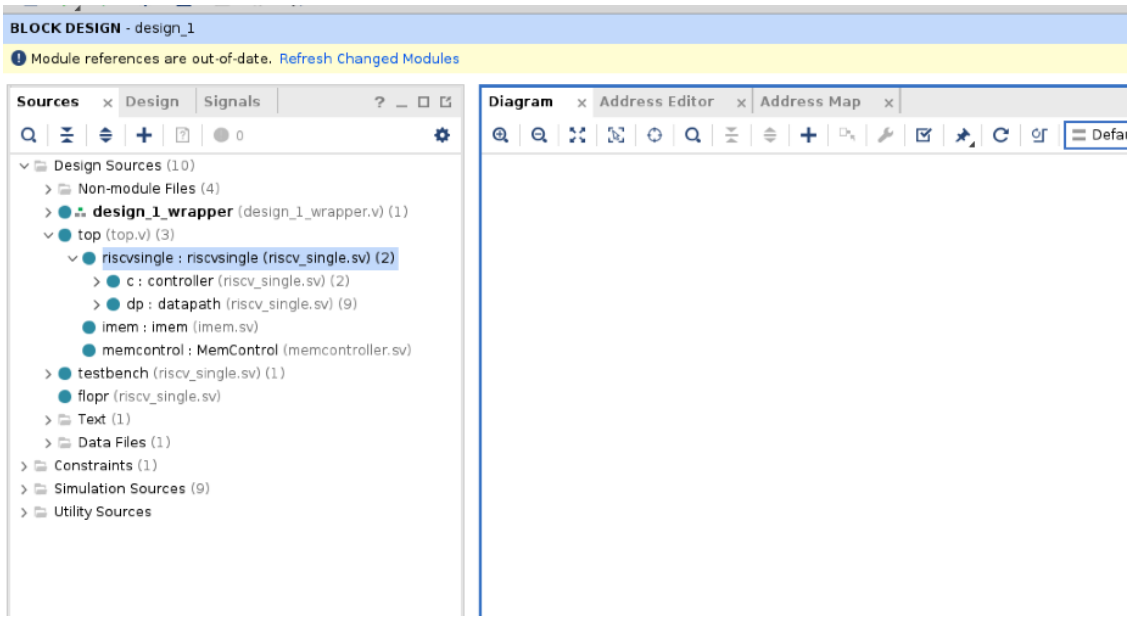

Figure 5: Refresh Changed Modules Banner

Power on the Elvis III using the power switch along the top-back of the device. Then, power on the expansion board with the button on the upper left of the Elvis III. Finally, make sure the boot switch `SW8` on the DSDB is set to `QSPI`, and then power on the DSDB with the power switch on the upper right of the board. Now your DSDB is powered on and connected to your system. You can check to make sure you have the correct boot switch setting by flipping the switches `SW0-SW7` on the board, and their corresponding LEDs `LD0-LD7` should light up.

8. With the Hardware Manager open in Vivado and your DSDB connected to the Elvis III and the computer as well as powered on, click Open Target along the top banner that alerts you that no hardware target is open, and then click Auto Connect (i.e., Figure 7). This will automatically search the system for any powered-on devices that can interface with Vivado and connect to them for easy programming.

9. Once connected, the auto connect banner along the top will change to tell you there are no debug cores connected. This isn't an issue - our debug core is built-in to the design so it will connect once you program the device. Click Program device.

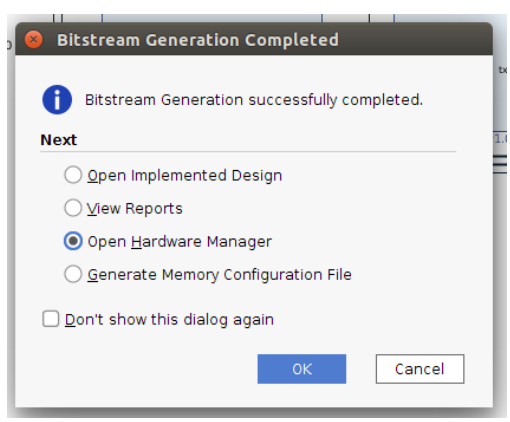

Figure 6: Bitstream Complete, Open Hardware Manager

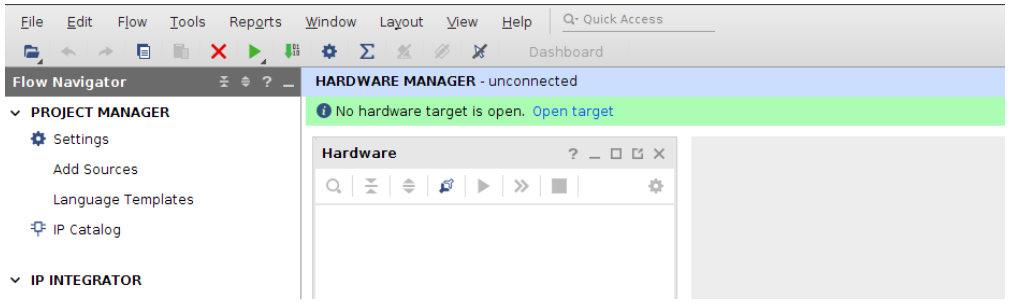

Figure 7: Open Hardware Target

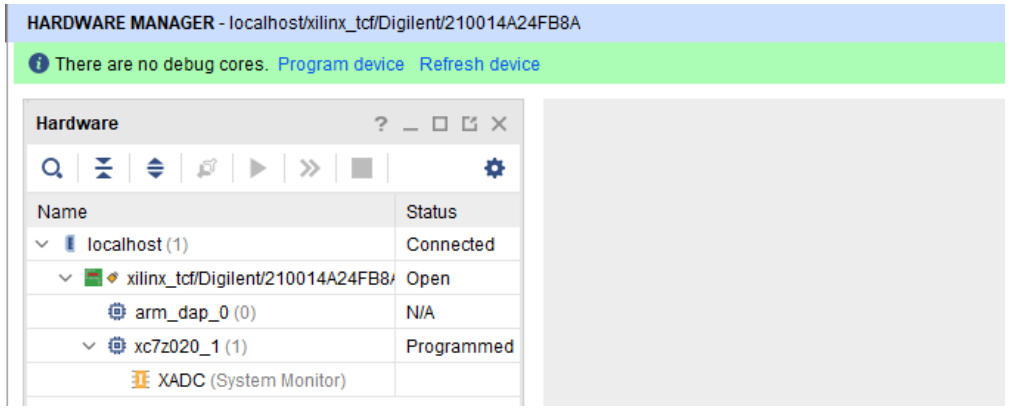

Figure 8: Program Device once Connected

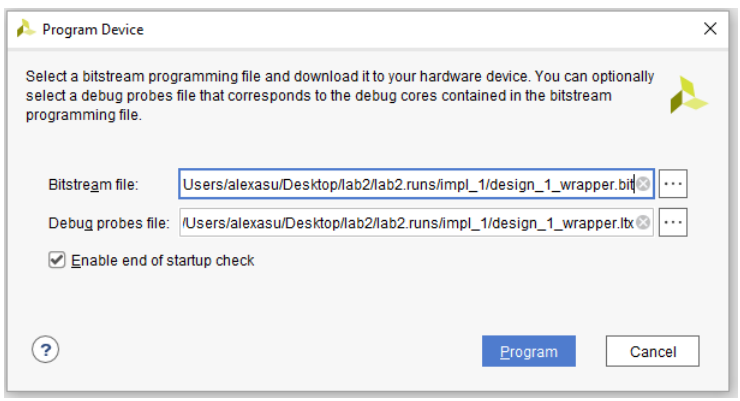

Figure 9: Select BitStream and Debug Image to Program

Starting at this stage and until specified later, you need to press and hold the bottom-right button `BTN0` on the DSDB. This button is bound to the reset signal in the processor and will keep the processor in a reset state once it has been programmed. We want to keep the processor in reset so it doesn't begin running until we've finished prepping the debug tool that will let us observe the system as it runs, so continue holding the button down while programming the device and prepping the debug window.

After clicking Program device, a window will open in which you get to select the bitstream and debug profile to upload to the FPGA (i.e., Figure 9). These fields will be auto-populated, so (with `BTN0` held down), you can click program FPGA. The FPGA will load your design and the Hardware Manager window will change to look like a signal analyzer similar to ModelSim.

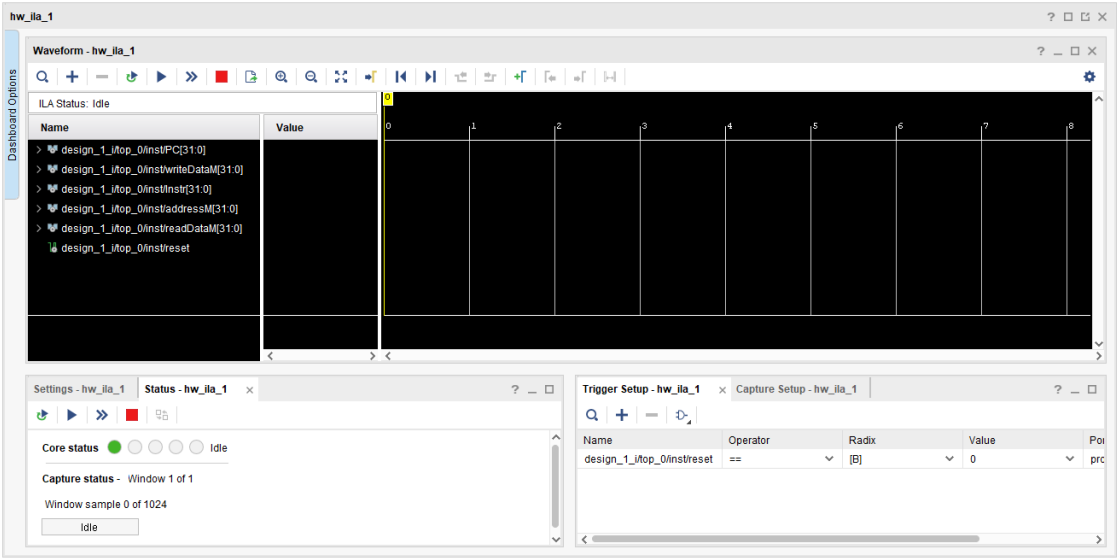

Figure 10: Signal Analyzer and Debug Trigger Menus

10. With the FPGA programmed and `BTN0` still held down, press the play button along the top panel of the signal analyzer view that has appeared in the Hardware Manager window to arm a debug trigger that has been pre-applied (i.e., Figure 10).

    Unlike simulations which can just generate the next value and save that on the computer, the FPGA is a real system with a real circuit on-board, meaning that capturing the data inside is a little trickier. To capture this data, a debug core has been pre-added to this project that observes some of the data that flows into and out of your RISC-V core. But, because these data measurements take place on the FPGA and then the data is sent back to the computer, only so much data can be saved at a time. This is where debug triggers come in - we can set the debug core to only save data when a certain event takes place such as a specific signal reaching a specified value or even a Boolean function of multiple signals. We have pre-made a trigger for this project to detect when the reset signal is logic low (which, because you are still holding down `BTN0`, is currently logic high). Pressing the play button arms the trigger so the debug core starts actively cycling through data, waiting to save the current data it has as soon as the trigger is true.

    Once the play button has been pressed and the trigger is armed, release `BTN0` which will complete the trigger requirements and cause the debug core to start taking data. This will simultaneously bring the RISC-V core out of the reset state and it will begin executing the program it was loaded with (the default is `memfile.dat`).

    The signal analyzer window will populate with data - 512 data points per signal before the reset signal went low and 512 data points per signal after the reset signal went low. You can use the values in the signal analyzer window to view view the performance of your system by matching the value of your RISC-V core's FPGA with the instruction it should be executing at that point and see as values pass by on the memory and address busses.

    Specifically, we are interested in the memory performance of the system, so we want to see how long the system is forced to wait for `LW` and `SW` operations to complete. `fib.s` has both of these operations, but you should test other programs later as well. Once you've found a memory operation (fairly easy to find, as they take the most time when looking at the signal analyzer view), calculate how many cycles they take by positioning the cursor (click and drag it) at the end of the operation to get the exact end time, then position the cursor at the start of the operation to get the exact start time. Subtract start time from end time and that will be your memory response time for this particular program on your RISC-V core.

Remember that load and store both take different amounts of time, so measure the response time of both. Of course, this is also a good opportunity to make sure your RISC-V core is still functional by seeing if the value read out of memory in the `memfile.dat` program matches the value that was written to memory just a couple of instructions prior. If that isn't working, then your measure of the memory's response time may not be accurate because the rest of the processor isn't working as intended.

If you re-run the trigger and press reset after you have already captured data once, the debug core will still capture data and your RISC-V core will still be functioning, but because the system repeats the same program over and over (the instruction memory in the FPGA is only as big as it needs to be to hold the program you specified to load in) the positions used in memory for the program will already hold the correct values and the values read from memory might still be on the memory read port due to the small number of `LW` instructions preventing the value on that port from being overwritten.

You can rerun the program from scratch by powering off the DSDB and then powering it back on. Vivado might throw an error because the board was disconnected without warning, but simply follow the steps to auto-connect to the board and proceed from there to run a completely fresh run again.

By now, you should have your Vivado project set up so that it contains your modified and functional `riscv_single.sv` file, the project has run through the toolchain including Block Generation, Synthesis, Implementation, and finally Bitstream Generation, and you've been able to upload the bitstream to the FPGA and see your processor function correctly on the FPGA running the `memfile.dat` program. With this, you've been able to take the data on how long each `LW` and `SW` operation should take when using real memory like that of the DSDB.

4. Adding your own Programs into Vivado

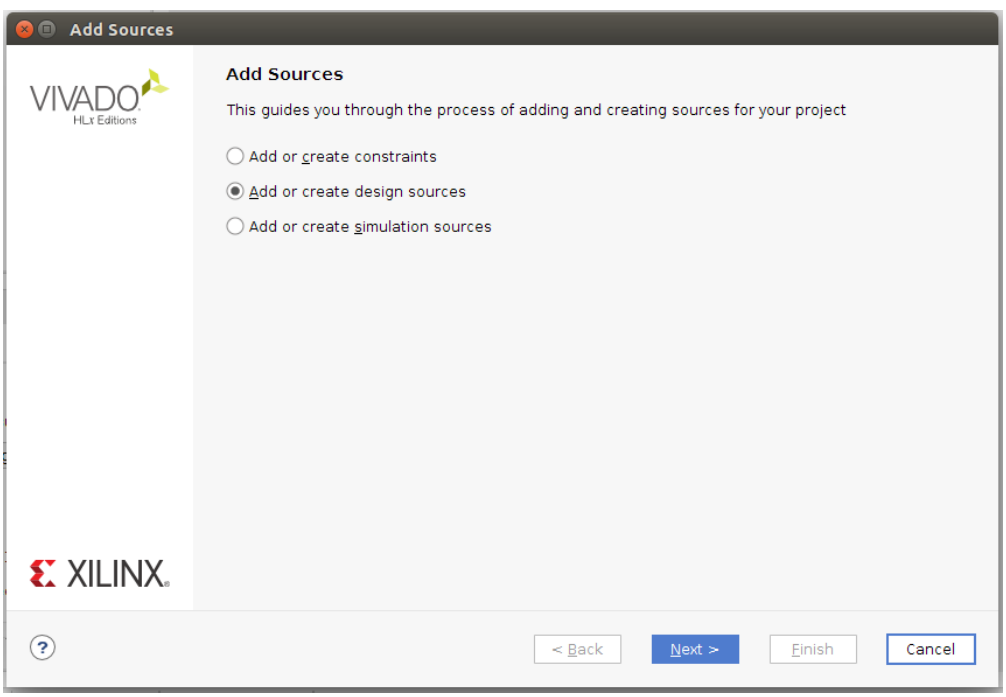

Figure 11: Open Hardware Target

You can run your own programs on your RISC-V core, as well. To do this, you will need to import the `.dat` file for the program into your Vivado project, then follow the steps in the previous section on Changing the Program the RISC-V Core will run to change which program will be loaded into the instruction memory. Right now, the instruction memory is hard-coded into the FPGA within the Block RAM.

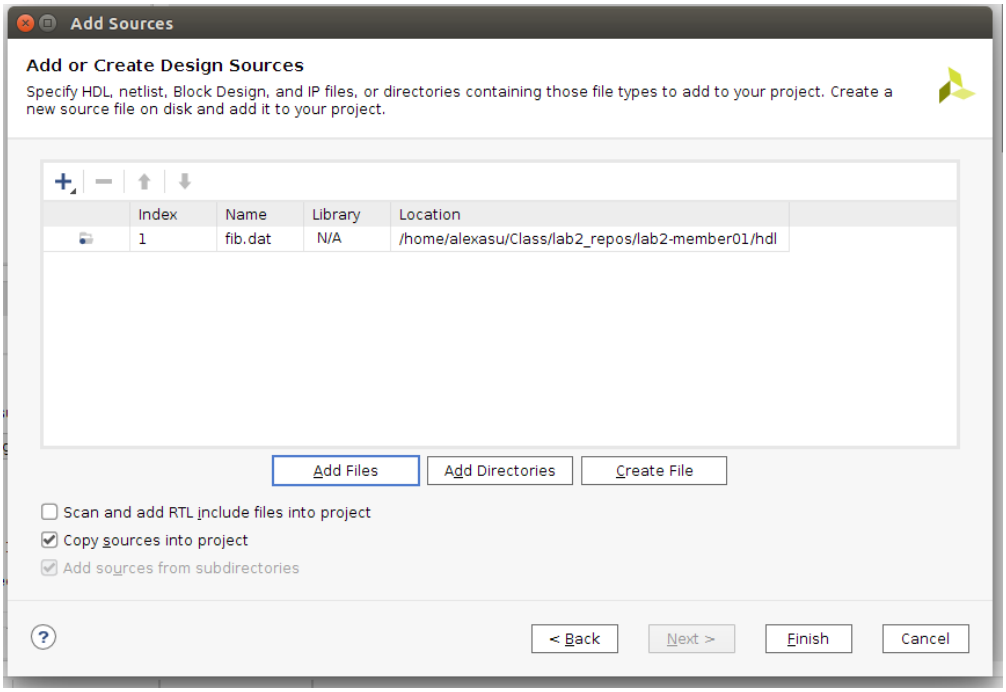

Figure 12: Open Hardware Target

Make sure you test your hardware with `test_hw.s` in the `risvtest` subdirectory of the repository. You should be able to compare the state of the hardware compared to what you get with `spike`. To test with spike, you can type: `spike --isa=rv32i -d test_hw.elf`. To get full credit for this lab, the state of your hardware should match.

1. Click Add Sources under the Project Manager heading on the left panel (i.e., Figure 4).
2. Select Add or create design sources on the window that opens and click Next (i.e., Figure 11).
3. Select Add Files to open a file browser you can use to navigate to your `.dat` file. By default, the file browser will be looking for HDL source files, so you'll need to change file type visibility by clicking the Files of Type drop down menu and selecting All Files. Select your `.dat` file and click OK. You can select more than one `.dat` file if you'd like to add multiple files at the same time.
4. The files you selected will be listed in the Add or Create Design Sources window. Make sure the Copy sources into project box is checked and then click Finish (i.e., Figure 12).
5. With the new `.dat` files added to your project, follow the steps in the previous section to change which `.dat` file will get loaded into the instruction memory and then continue through the steps to run the toolchain, upload the bitstream to the FPGA, and test your design.

References

[1] "User Manual: NI Digital System Development Board," National Instruments, Tech. Rep. Pub ID: 376627B-01, 2018.